AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 12.01.2027

Lab 13

Sortowanie, wyszukiwanie, notacja Big O 🤖

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć, **jak mierzyć koszt algorytmu** niezależnie od komputera
- Sortowanie bąbelkowe, przez wybieranie (i przez scalanie ☆)
- Wyszukiwanie liniowe i binarne
- Sprawdzić teorię pomiarem i ocenić algorytmy wygenerowane przez AI 🤖
- Efekty z sylabusa: **U3** (sortowanie i wyszukiwanie), **W2**, **KS1**



CZĘŚĆ I

Teoria: złożoność obliczeniowa

Notacja Big O

> **Definicja.** $f(n) = O(g(n))$, jeśli istnieją stałe $c > 0$ i n_0, takie że $f(n) \leq c \cdot g(n)$ dla wszystkich $n \geq n_0$.

- Opisuje, **jak rośnie** liczba operacji wraz z rozmiarem danych n – pomija stałe i składniki niższego rzędu
- $3n^2 + 5n + 7 = \mathbf{O(n^2)}$

N	$\log_2 N$	N	$N \log_2 N$	N^2
10	3	10	33	100
1 000	10	1 000	9 966	1 000 000
1 000 000	20	1 000 000	~20 mln	10^{12}

- Przy 10^9 prostych operacji na sekundę: n^2 dla miliona elementów to ok. **17 minut**, n log n – ułamek sekundy

Typowe klasy złożoności

KLASA	NAZWA	PRZYKŁAD
O(1)	stała	`lst[i]`, `d[key]`
O(log n)	logarytmiczna	wyszukiwanie binarne
O(n)	liniowa	wyszukiwanie liniowe, `sum(lst)`, `x in lst`
O(n log n)	liniowo-logarytmiczna	`sorted()`, merge sort
$O(n^2)$	kwadratowa	bąbelkowe, przez wybieranie
$O(2^n)$	wykładnicza	naiwny rekurencyjny Fibonacci (lab 07)

- Analizujemy zwykle **najgorszy przypadek**; ważna jest też **złożoność pamięciowa**

Sortowanie bąbelkowe i przez wybieranie

Bąbelkowe: porównuj sąsiadów, zamieniaj; największe elementy „wypływają” na koniec

```
[5, 3, 8, 1] → [3, 5, 1, 8] → [3, 1, 5, 8] → [1, 3, 5, 8]
```

Przez wybieranie: znajdź minimum reszty i postaw je na kolejnej pozycji

```
[5, 3, 8, 1] → [1, 3, 8, 5] → [1, 3, 8, 5] → [1, 3, 5, 8]
```

ALGORYTM	NAJLEPSZY	NAJGORSZY	PAMIĘĆ	STABILNY
bąbelkowe (z flagą)	O(n)	$O(n^2)$	O(1)	tak
przez wybieranie	$O(n^2)$	$O(n^2)$	O(1)	nie
przez wstawianie	O(n)	$O(n^2)$	O(1)	tak
przez scalanie	O(n log n)	O(n log n)	O(n)	tak

Stabilność i sortowanie po wielu kluczach

- **Stabilne** sortowanie zachowuje kolejność elementów o równych kluczach
- `sorted()` w Pythonie jest stabilne → sortowanie wielokryterialne:

```
sorted(books, key=lambda b: (b["author"], b["year"]))     # krotka jako klucz
sorted(books, key=lambda b: b["pages"], reverse=True)     # malejąco
```

- Każdy algorytm sortujący przez **porównania** potrzebuje w najgorszym przypadku Ω(n log n) porównań – szybciej się nie da (dolna granica)

Wyszukiwanie liniowe a binarne

```python
def binary_search(a, target):          # a – POSORTOWANA
    low, high = 0, len(a) - 1
    while low <= high:
        middle = (low + high) // 2
        if a[middle] == target:
            return middle
        if a[middle] < target:
            low = middle + 1           # szukaj w prawej połowie
        else:
            high = middle - 1          # szukaj w lewej połowie
    return -1
```

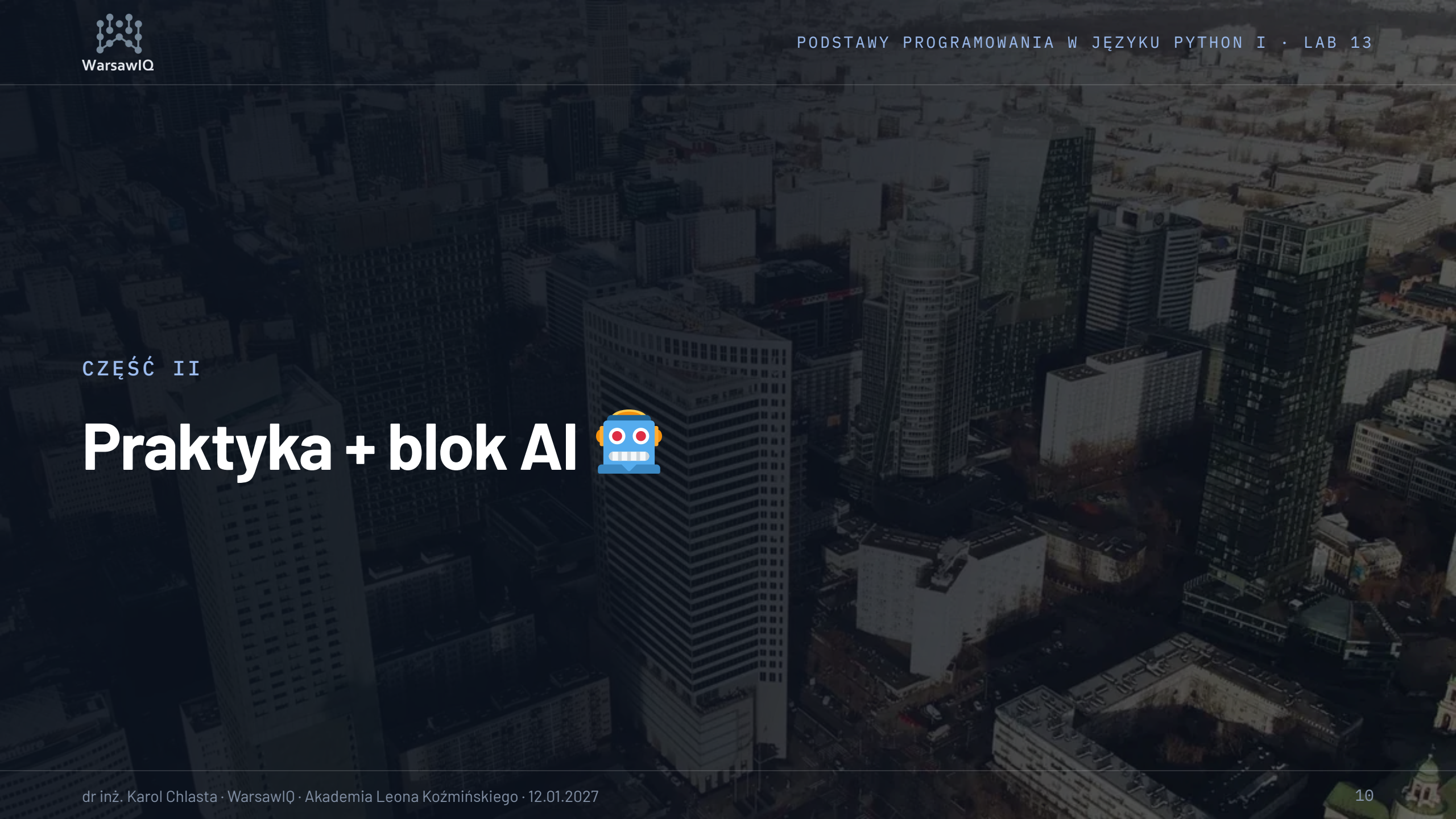
ELEMENTÓW	LINIOWE (NAJGORZEJ)	BINARNE (NAJGORZEJ)
1 000	1 000	10
1 000 000	1 000 000	20

Teoria vs pomiar

```
start = time.perf_counter()
bubble_sort(data)
elapsed = time.perf_counter() - start
```

JEŚLI N ROŚNIE 2×	O(N)	O(N LOG N)	O(N²)
czas rośnie ok.	2×	trochę ponad 2×	**4×**

- Pomiar zależy od komputera, obciążenia, pamięci podręcznej – Big O mówi o **trendzie**
- `sorted()` jest napisane w C i jest szybsze o rzędy wielkości – w praktyce **zawsze używaj `sorted()`**; własne sortowanie piszemy, żeby zrozumieć, jak to działa



CZĘŚĆ II

Praktyka + blok AI 🤖

Blok AI: algorytm od AI pod lupą

1. Poproś AI o **quicksort** w Pythonie
2. Przetestuj na: `[]`, `[1]`, duplikatach, liście posortowanej i malejącej
3. Poproś o **złożoność** – i sprawdź, czy AI wspomniało o najgorszym przypadku $O(n^2)$ (zły wybór elementu dzielącego)
4. Zapytaj o złożoność Timsorta – i zweryfikuj w dokumentacji
5. Zapisz wszystko w `AI.md`

> Kod algorytmów od AI często działa „na przykładzie z promptu”, a zawodzi na przypadkach brzegowych. Twoje testy są ważniejsze niż pewny ton odpowiedzi.

Ciekawostki 🔍

- **Jon Bentley** (*Programming Pearls*): gdy poprosił zawodowych programistów o wyszukiwanie binarne, tylko ok. **10%** napisało je bezbłędnie
- **Joshua Bloch** (Google, 2006): przez lata `(low + high) / 2` w Javie przepełniało typ `int` dla ogromnych tablic. W Pythonie to nie grozi – `int` nie ma limitu
- Sortowanie przez scalanie opisał **John von Neumann** w 1945 r.
- 2007 r., Google: na pytanie, jak posortować milion liczb 32-bitowych, **Barack Obama** odpowiedział, że *„sortowanie bąbelkowe byłoby złą drogą”*
- Od Pythona 3.11 Timsort scala serie według strategii **Powersort** (Munro, Wild)

Dzisiejsze zadania

	PLIK	ZADANIE
★	`zad1_sortowania.py`	bąbelkowe i przez wybieranie
★	`zad2_wyszukiwanie.py`	liniowe i binarne
★	`zad3_biblioteka.py`	sortowanie biblioteki
★	`zad4_pomiar.py`	eksperyment z czasem
🤖	`AI.md`	quicksort od AI pod lupą
☆	`zad5_szybsze.py`	wstawianie i scalanie

Sprawdzenie i oddanie

```
cd lab13_sortowanie
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 13"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
- 📝 **Kolokwium II: wtorek 26.01, 08:30–11:45, sala A/111** – zakres: laby 09–14

Do poczytania

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Wprowadzenie do algorytmów*, PWN
- J. Bentley, *Perełki oprogramowania*, Helion
- Dokumentacja: *Sorting Techniques*, moduł `bisect`
- Wizualizacje algorytmów: *visualgo.net*

Następny lab (19.01, sala A/111): testy jednostkowe i TDD + blok AI 🤖